1. Распределённая система. Причины создания, принципы построения, последствия распределённости, требования к распределённым системам.

Распределенная система – это собрание независимых компьютеров (узлов), соединенных сетью и программным обеспечением, обеспечивающим их совместное функционирование.

Причины создания распределенных приложений

1. **Необходимость совместного использования общих ресурсов** (данных, устройств, приложений).
2. **Улучшение функциональных характеристик системы** (производительности, повышения надежности и устойчивости к сбоям, упрощение и удешевление за счет специализации компонентов, отношения цена/производительность).

Принципы построения

1. **Функциональное разделение.** Узлы выполняют различные задачи (клиент / сервер, хост / терминал, сбор данных / обработка данных). Решение – создание разделяемых сервисов, которые управляют набором ресурсов и предоставляют услуги пользователям.
2. **Естественное разделение.** Разделение определяется задачей (система обслуживания сети супермаркетов, сеть для обеспечения коллективной работы). Функциональная нагрузка одинаковая у однотипных узлов.
3. **Балансировка нагрузки.** Функциональность может повторяться. Задачи назначаются на процессоры таким образом, чтобы нагрузка была равномерной. Такой подход обеспечивает наибольшую эффективность использования вычислительных ресурсов.

Последствия распределенности

1. **Параллельность.** Параллельное выполнение (вопросы синхронизации, «гонки потоков»), совместное использование ресурсов (коллизии при

доступе к данным, сервисам, устройствам), типичные проблемы (взаимные блокировки, ненадежные коммуникации).

2. **Нет “глобального” времени** (асинхронная передача сообщений, ограниченная точность синхронизации часов).
3. **Нет состояния системы.** В распределенной системе нет ни одного процесса, который бы знал текущее глобальное состояние системы. Это следствие параллелизма и механизма передачи данных.
4. **Сбои.** Процессы выполняются автономно, изолированно. Неудачи отдельных процессов могут остаться необнаруженными. Отдельные процессы могут не подозревать об общесистемном сбое. Сбои происходят чаще, чем в централизованной системе. Новые причины сбоев (которых не было в монолитных системах). Сетевые сбои изолируют процессы и фрагментируют систему на изолированные части.

Требования к распределенным системам

1. Открытость.
2. Безопасность.
3. Масштабируемость.
4. Механизмы обработки ошибок и восстановления после сбоев.
5. Методы решения проблем параллелизма.
6. Прозрачность.
7. Управляемость.

1. **Открытость.** Гарантирует расширяемость, возможность повторного использования, использование открытых протоколов и стандартов. Важными факторами являются наличие четких спецификаций и полной документации, опубликованные интерфейсы, тестирование и проверка на многих платформах.
2. **Безопасность.** Физическая распределенность означает возможность доступа злоумышленников к компонентам. Необходимо обеспечить посылку важной информации по сети безопасно и эффективно. Это достигается, например, с помощью аутентификации (процедуры проверки

подлинности данных) и авторизации (предоставление определённому лицу или группе лиц прав на выполнение определённых действий; а также процесс проверки данных прав при попытке выполнения этих действий), шифрования данных.

3. **Масштабируемость.** Распределенная система масштабируема, если она остается эффективной при увеличении числа обслуживаемых пользователей или ресурсов. Возникает проблема стоимости физических ресурсов (растет при увеличении числа пользователей) и потери производительности (увеличиваются с ростом размера данных и количества пользователей). Для «обхода узких мест» используется децентрализация алгоритмов (Пример – Domain Name Service (DNS)), тиражирование и кэширование данных.
4. **Обработка сбоев.** Сбои возникают чаще, чем в централизованных системах, но обычно носят локальный характер. Обработка сбоев включает в себя: диагностику (например, ошибки передачи могут быть обнаружены с помощью контрольных сумм. Однако, в ряде случаев, может быть невозможна. Например, невозможно определить, удаленный сервер не работает или просто очень загружен?), маскирование (многие сбои могут быть скрыты, однако, в ряде случаев маскирование может быть невозможно или не всегда приводит к правильным последствиям) и восстановление.
5. **Прозрачность.** Это сокрытие гетерогенной (разнородной) и распределенной структуры системы таким образом, чтобы пользователю система представлялась монолитной. Степень прозрачности взаимосвязана с производительностью. Различают:
   - **Прозрачность доступа.** Скрывается разница в представлении данных и доступе к ресурсам.
   - **Прозрачность расположения.** Скрывается местоположения ресурса.
   - **Прозрачность переноса.** Скрывается факт перемещения ресурса в другое место.

- **Прозрачность смены местоположения.** Скрывается факт перемещения ресурса в другое место в процессе обработки.
- **Прозрачность репликации.** Скрывается факт и особенности репликации (механизм синхронизации содержимого нескольких копий объекта (например, содержимого базы данных)) ресурса.
- **Прозрачность параллелизма.** Возможность нескольким процессам параллельно работать с ресурсами, не оказывая влияния друг на друга.
- **Прозрачность обработки ошибок.** Защита программных компонентов от сбоев, произошедших в других программных компонентах; восстановление после сбоев.

  Критически важными являются:
  - Прозрачность доступа.
  - Прозрачность расположения.
  - Прозрачность паралеллизма.

6. **Управляемость.** Распределенные ресурсы не имеют центральной точки управления. Локальная оптимизация не всегда означает глобальную оптимизацию (нужно учитывать гетерогенность и другие особенности, нужен глобальный взгляд на проблему, он не всегда возможен (есть системы, никому конкретно не принадлежащие)).

2. Распределённая система. Сложности при реализации, модели распределённых систем, архитектура и основные принципы, преодоление сложности.

Распределенная система – это собрание независимых компьютеров (узлов), соединенных сетью и программным обеспечением, обеспечивающим их совместное функционирование.

Сложности при реализации

1. **Выбор архитектуры.** Важнейшие характеристики системы очень сильно зависят от выбранной архитектуры. Практически невозможно отказаться от принятых в начале проектирования неверных решений. **Решение:** использование широко известных шаблонов (образцов) архитектур.
2. **Гетерогенность среды (разнородность).** Различаются:
    - Сетевые инфраструктуры.
    - Оборудование (например, Intel и Motorolla).
    - Программное обеспечение (например, UNIX sockets и Winsock calls).
    - Языки программирования.
    - Представления данных.

    Различные компоненты системы выполняются на различных платформах. Различия должны быть скрыты. Интерфейсы и реализация могут быть разными, но базовые концепции обычно неизменны. Средства борьбы с гетерогенностью – стандарты.

    Решения:

    - Использование распространенных открытых стандартов и протоколов.
    - Использование промежуточного программного обеспечения (middleware).

    Middleware: промежуточный программный слой:

    - Позволяет гетерогенным узлам взаимодействовать.
    - Определяет однородную вычислительную модель.
    - Поддерживает один или несколько языков программирования.
    - Обеспечивает поддержку распределенных приложений (вызов удаленных объектов, удаленный вызов SQL, распределенную обработку транзакций).

    Примеры: CORBA, JavaEE, Microsoft DCOM.
3. **Сложность развертывания.**

- Фрагментация (разделение приложения на модули развертывания).
- Конфигурация (связь модулей друг с другом (зависимости)).
- Размещение (выгрузка модулей в целевую систему, распределение вычислительных модулей между узлами (статическое или динамическое), использование специализированных инструментальных средств).

4. **Сложность отладки.** Нет глобального состояния. Параллельность приводит к неповторяемости (невоспроизводимости) результатов выполнения. Компоненты распределены по разным узлам.

Модели распределенных систем

1. **Модели архитектуры.** Модель архитектуры распределенной системы должна содержать решение двух проблем: физическое размещение компонентов между узлами и взаимодействие между компонентами. Реальные функции отдельных компонентов не указываются, указываются:
    - Расположение (размещение по узлам).
    - Шаблоны распределения данных и задач по их обработке.
    - Взаимодействие компонентов (роли компонентов, шаблоны взаимодействия).
2. **Другие модели.** Формальное описание различных параметров и свойств системы, модели взаимодействия, обработки ошибок, безопасности, и т.д.

Архитектура

- Определяет разделение системы на наиболее крупные составные части.
- Определяет конструктивные решения, которые после их принятия с трудом поддаются изменению.
- Отображает общий взгляд разработчиков на результаты проектирования системы: идентификация главных компонентов системы, способов их взаимодействия, выбор основополагающих решений, не подлежащих изменению в будущем.

Основные принципы архитектуры

- **Согласованность.** Частичное знание системы позволяет предсказать остальное.
- **Ортогональность.** Функции независимы и специфицированы по отдельности.
- **Соответствие.** Включаются только функции, соответствующие существенным требованиям к системе, нет ненужных функций.
- **Экономичность.** Отсутствие дублирования.
- **Прозрачность.** Функции должны быть известны пользователю.
- **Общность.** Если функция должна быть введена, ее следует вводить в таком виде, чтобы она отвечала как можно большему числу назначений.
- **Открытость.** Можно использовать функцию иначе, чем это предполагалось при проектировании.
- **Полнота.** Введенные функции должны с учетом экономических и технологических ограничений как можно полнее соответствовать требованиям и пожеланиям пользователя.

Преодоление сложности:

1. **Использование типовых решений - образцов проектирования.** Каждое типовое решение описывает некую повторяющуюся проблему и ключ к ее разгадке, причем таким образом, что вы можете пользоваться этим ключом многократно, ни разу не придя к одному и тому же результату.

Структура типовых решений

- Название решения.
- Назначение (аннотация).
- Мотивация, применимость.
- Принцип действия (структура, участники, отношения).
- Результаты.
- Реализация.

2. **Разделение системы на слои (расслоение).** Основная идея: независимость нижележащих уровней от вышележащих. Основная задача: уменьшать сложность систем, разделяя их на слои и сервисы:
    - **Слой (уровень):** группа сильно связанных и закрытых элементов, реализующих одну функциональность.
    - **Сервис:** функциональность, обеспечиваемая для вышестоящего слоя.

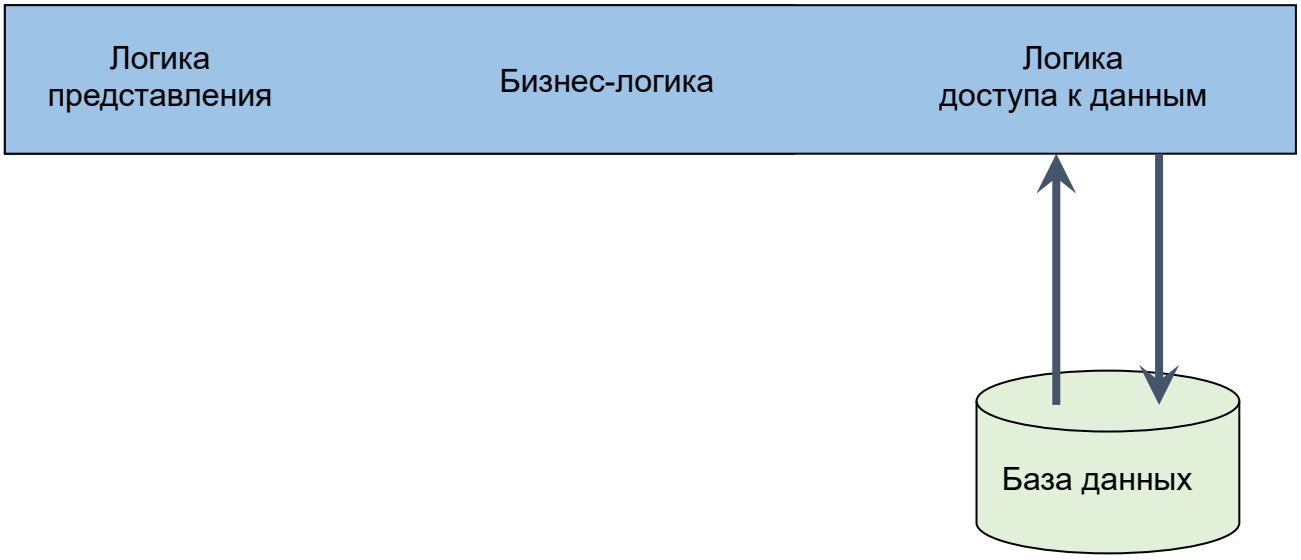


Рисунок 22 – Монолитная система

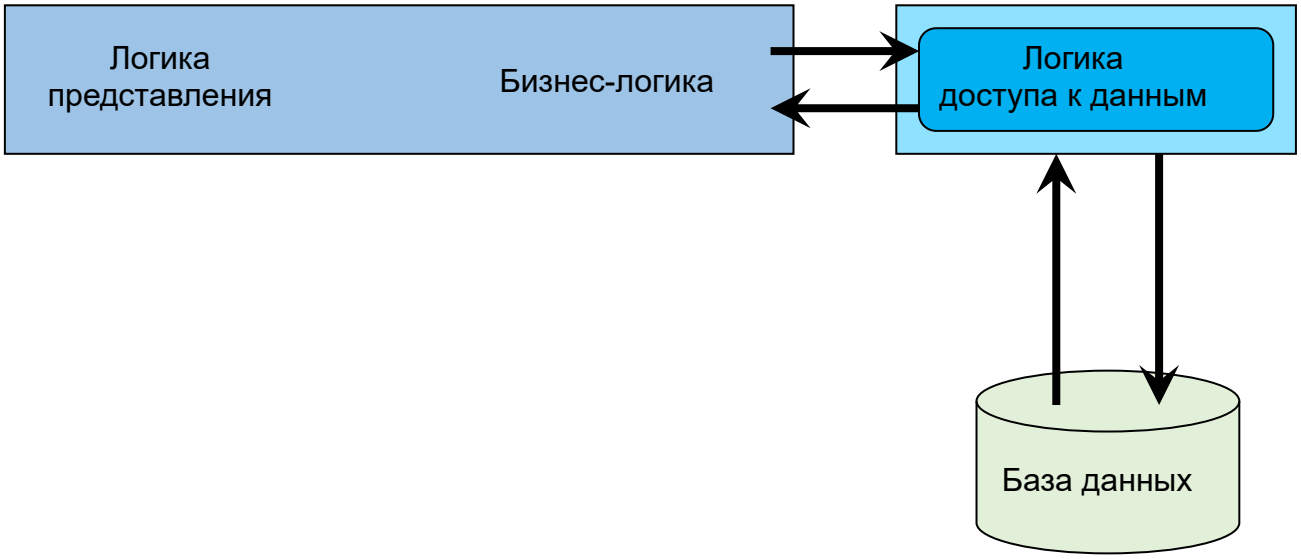


Рисунок 23 – Двухслойная система

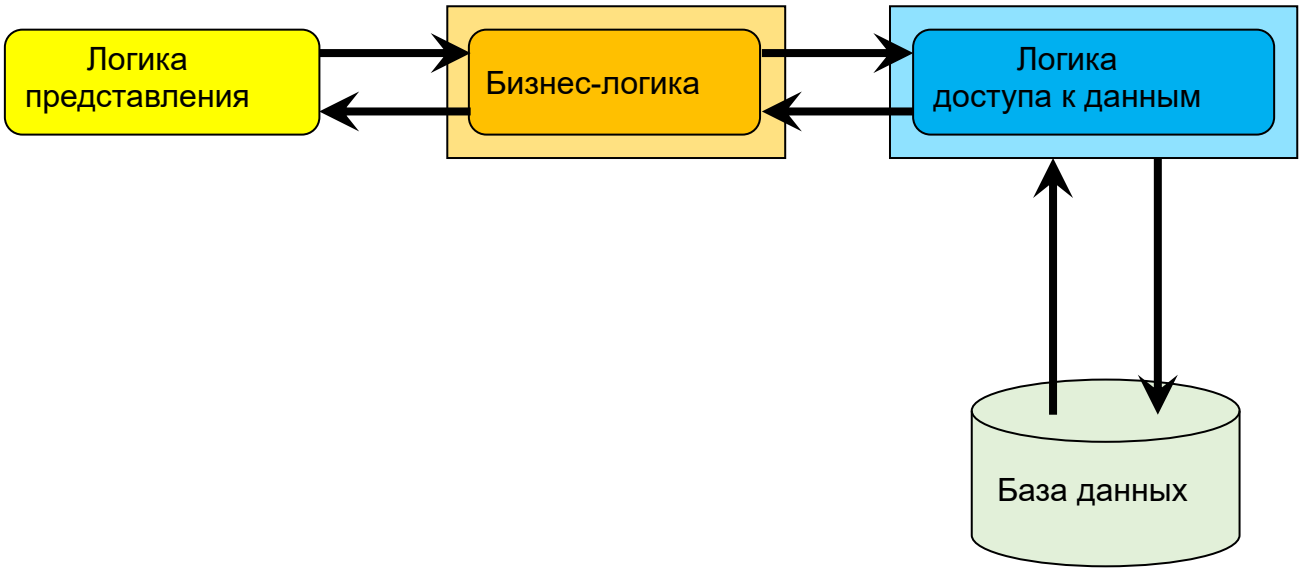


Рисунок 24 – Трехслойная система

Три основных слоя

- Слои:
  - представление (presentation).
  - домен (domen) – предметная область, бизнес-логика.
  - работа с данными (data source).
- Рекомендуется различать:
  - **Слой** (layer) – логическое разделение.
  - **Уровень** или **Ярус** (tier) – физическое разделение.

Итоги

- Распределенная система – это автономные (но соединенные средой передачи данных) узлы. Взаимодействие осуществляется посредством передачи сообщений.
- Много доводов в пользу того, что распределенные системы нужны и их нужно уметь строить.
- Распределенные системы существуют и их нужно уметь развивать и поддерживать.
- При разработке распределенных систем возникают специфические проблемы.

- Используется модель слоев для снижения сложности системы. Middleware обеспечивает дополнительное удобство и дополнительные сервисы.
- Выбор модели архитектуры зависит от особенностей задачи.
    - Клиент – сервер.
    - Модель предоставления услуг пулом серверов.
    - Модель прокси-и кэш-серверов.
    - Модель равных процессов.

3. JDBC. Типы драйверов. Основные цели интерфейса JDBC. Примеры использования JDBC. Основы программирования JDBC. Предварительно подготовленные команды. Метаданные.

Разработчики приложений используют JDBC API, а поставщики БД и инструментальных средств – JDBC Driver API. Подход, используемый при создании JDBC, основан на модели ODBC(Open Database Connectivity) — интерфейса доступа к БД, разработанного фирмой Microsoft: программы, соответствующие API, могут взаимодействовать с диспетчером драйверов JDBC, который, в свою очередь, использует присоединенные драйверы для организации взаимодействия с БД.

Типы JDBC – драйверов:

1. **Драйвер типа 1.** Транслирует JDBC в ODBC и для взаимодействия с БД использует драйвер ODBC. Sun включила в состав JDK один такой драйвер – мост JDBC/ODBC. Но для его использования требуется соответствующим образом установить и конфигурировать ODBC драйвер. В первом выпуске JDBC этот мост предполагалось использовать только для тестирования, а не для рабочего применения. В настоящее время имеется большое количество более удачных драйверов.
2. **Драйвер типа 2.** Создается преимущественно на Java и частично на собственном языке программирования, который используется для взаимодействия с клиентским API БД. Для использования

такого драйвера нужно помимо библиотеки Java установить специфический для данной платформы код.

3. **Драйвер типа 3.** Создается только на основе библиотеки Java, в которой используется независимый от БД протокол взаимодействия сервера и базы. Этот протокол позволяет транслировать запросы в соответствии со спецификой конкретной базы. Если код, зависящий от БД, находится только на сервере, доставка программ существенно упрощается.
4. **Драйвер типа 4.** Представляет собой библиотеку Java, которая транслирует JDBC-запросы непосредственно в протокол конкретной БД.

Основные цели интерфейса JDBC можно сформулировать следующим образом:

1) Разработчики создают программы на языке Java, пользуясь для доступа к БД стандартными средствами языка SQL, при этом они следуют только соглашениям языка Java.

2) Производители СУБД и инструментальных средств поставляют драйверы только низкого уровня.

Примеры использования JDBC

Согласно традиционной модели клиент/сервер, графический пользовательский интерфейс реализуется на стороне клиента, а БД располагается на стороне сервера. В этом случае драйвер JDBC находится на стороне клиента(рис.25).



Рисунок 25 – Традиционная структура приложения клиент/сервер

Однако современная тенденция развития программного обеспечения заключается в переходе от архитектуры клиент/сервер к «трехуровневой модели» (рис.26).

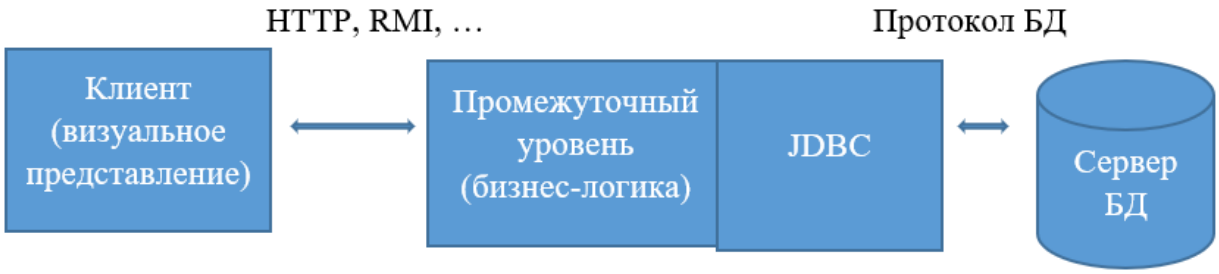


Рисунок 26 – Приложение, созданное в соответствии с трехуровневой моделью

В ней клиент не формирует обращений к БД. Вместо этого он обращается к средствам промежуточного уровня на сервере, который, в свою очередь, выполняет запросы к БД. Трехуровневая модель отделяет визуальное представление (на компьютере-клиенте) от бизнес-логики (промежуточный уровень) и данных (в базе данных). Таким образом, становится возможным доступ к тем же данным и тем же бизнес-правилам посредством клиентов различных типов, например, Java-приложений, апплетов и Web-форм.

Взаимодействие между клиентом и промежуточным уровнем может быть реализовано на основе протокола HTTP (при использовании Web-браузера в качестве клиента), средств RMI (при использовании приложений или апплетов) или с помощью какого-либо другого механизма. JDBC используется для управления взаимодействием между промежуточным уровнем и БД.

Основы программирования JDBC

Классы, используемые для программирования JDBC, содержатся в пакетах java.sql и javax.sql.

Класс *DriverManager* отвечает за выбор драйверов БД и создание нового соединения с БД. Однако перед тем, как диспетчер активизирует драйвер, его надо зарегистрировать. Имя драйвера можно задать в командной строке:

java –Djdbc.drivers=org.postgresql.Driver MyClass

Кроме того, приложение может установить системное свойство с помощью вызова:

System.setProperty("jdbc.drivers", "org.postgresql.Driver");

Драйвер также можно зарегистрировать вручную, загрузив его класс:

Class.forName("org.postgresql.Driver");

Такой подход применяют, если диспетчер не может загрузить драйвер из-за ограничений конкретного драйвера либо в том случае, когда программа выполняется в среде контейнера, например, сервлета.

После регистрации драйвера можно установить соединение с БД. При работе с JDBC база данных представляется с помощью URL (Uniform Resource Locator). Для PostgreSQL он принимает одну из следующих форм:

- *jdbc:postgresql:database*
- *jdbc:postgresql://host/database*
- *jdbc:postgresql://host:port/database*

Для подключения к базе нужно использовать класс Connection:

Connection connection = DriverManager.getConnection(url, username, password);

Например:

Connection connection = DriverManager.getConnection("jdbc:postgresql://localhost:5432/MyTest", "postgres", "root");

Для указания драйвера, URL, имени пользователя и пароля удобно применять файл свойств:

Пример 49

jdbc.drivers = org.postgresql.Driver

jdbc.url = jdbc:postgresql://localhost:5432/MyTest

jdbc.username = postgres

jdbc.password = root

//Далее читаем файл свойств и устанавливаем соединение с БД:

Properties props = new Properties();

FileInputStream in = new FileInputStream("database.properties");

props.load(in);

in.close();

String drivers = props.getProperty("jdbc.drivers");

if(drivers != null) System.setProperty("jdbc.drivers", drivers);

String url = props.getProperty("jdbc.url");

String username = props.getProperty("jdbc.username");

String password = props.getProperty("jdbc.password");

Connection connection = DriverManager.getConnection(url, username, password);

Метод *getConnection()* возвращает объект *Connection*, с помощью которого создаётся объект *Statement,* который используется для выполнения SQL – команд. Метод *executeQuery()* возвращает объект *ResultSet,* который можно использовать для построчного просмотра результатов.

Пример 50

//Извлечение данных:

Statement statement = connection.createStatement();

*ResultSet resultSet = statement.executeQuery("SELECT * FROM item");*

while (resultSet.next()) {

String str = resultSet.getString("name");

System.out.println(str);

}

В классе *ResultSet* итератор располагается в начале работы перед первой строкой, поэтому для перемещения его к первой строке нужно вызвать метод *next()*. Строки в наборе результатов располагаются совершенно произвольно. Если порядок следования важен, его необходимо установить с помощью оператора ORDER BY.

При обработке каждой отдельной строки нужно с помощью специальных методов получить содержимое каждого столбца.

resultSet.getString(1);

resultSet.getDouble("Price");

Для каждого типа данных языка Java предусмотрен отдельный метод извлечения информации, например, *getString() и getDouble().* Для каждого из них реализовано два варианта, один из которых предполагает числовой, а второй – строковый параметр. При использовании числового параметра метод извлечет данные из столбца с указанным номером, нумерация столбцов начинается с 1. При использовании строкового параметра метод извлечет данные из столбца с указанным именем.

Если указанный тип не соответствует фактическому типу, метод извлечения данных выполняет преобразование. Например, метод *resultSet.getString("Price")* преобразует число с плавающей точкой из столбца Price в строку.

Типы данных SQL и Java не всегда совпадают. В таблице 3 приведен список основных типов данных языка SQL и их эквивалентов в Java.

Таблица 3

Типы данных языка SQL	Типы данных языка Java
INTEGER или INT	int
SMALLINT	short
NUMERIC(m,n), DECIMAL(m,n) или DEC(m,n)	java.math.BigDecimal
REAL	float
DOUBLE	double
CHARACTER(n) или CHAR(n), VARCHAR	String
BOOLEAN	boolean
DATE	java.sql.Date
TIME	java.sql.Time

Метод *executeUpdate()* возвращает количество строк, полученных в результате выполнения SQL-команды. Он может применяться для выполнения команд INSERT, UPDATE и DELETE, а также команд определения данных, в частности, CREATE TABLE и DROP TABLE.

Закончив работу с классом ResultSet, Statement или Connection нужно вызвать метод *close(),* так как перечисленные объекты используют структуры данных большого объема и не стоит ждать, пока процедура сборки мусора освободит ресурсы. Метод *close()* класса Statement автоматически закрывает связанные с ним наборы результатов(если они открыты). Аналогично, метод *close()* класса Connection закрывает все объекты Statement для этого соединения. Если время существования соединения невелико, нужно поместить вызов метода *close()* в блок finally, гарантируя, что объект соединения не останется открытым.

Предварительно подготовленные команды

Вместо создания отдельной SQL-команды для каждого запроса следует предварительно подготовить запрос с подстановочной переменной и многократно использовать его, меняя только значение переменной. Эта возможность существенно повышает эффективность работы программы. Перед каждым выполнением запроса СУБД создает план его эффективного исполнения. Предварительно подготавливая запрос для последующего многократного повторного использования, можно избежать повторного создания плана. Подстановочная переменная в запросе обозначается символом ?. При использовании нескольких подстановочных переменных нужно следить за их расположением, чтобы правильно выполнять подстановку фактических значений.

```
String s = "SELECT name, price FROM tovar WHERE price >? AND price <?";
PreparedStatement ps = connection.prepareStatement(s);
ps.setDouble(1,2000);
ps.setDouble(2,6000);
ResultSet rs1 = ps.executeQuery();
```

Перед выполнением предварительно подготовленного запроса нужно с помощью метода *set()* связать подстановочные переменные с их фактическими значениями. Как и при использовании методов *get()*, класса *ResultSet,* для различных типов предусмотрены разные методы *set()*. Например:

ps.setDouble(1,2000);

Первый параметр обозначает номер позиции подстановочной переменной, второй – её фактическое значение. При повторном использовании предварительно подготовленного запроса с несколькими подстановочными переменными все связи остаются в силе, если только они не изменены с помощью метода *set()*. Это значит, что методы *set()* нужно вызывать только для тех подстановочных переменных, которые изменяются в последующих запросах. Для изменения данных используется метод *executeUpdate()*.

Метаданные

В JDBC предусмотрены дополнительные возможности для получения информации о структуре таблиц и самой БД. Например, можно получить список всех таблиц БД либо имена всех столбцов с типами данных в них. Эти сведения пригодятся, если вы пишете программный продукт для работы с произвольной БД.

В языке SQL информация о структуре БД и её компонентов называется **метаданными**. Информация о структуре БД хранится в классе DatabaseMetaData. Ниже приведен пример получения набора результатов, содержащего информацию обо всех таблицах БД.

DatabaseMetaData meta = connection.getMetaData();

ResultSet mrs = meta.getTables(null, null, null, new String[] {"TABLE"});

Метод *getTables(String catalog, String schemaPattern, String tableNamePattern String types[])* представляет описание всех таблиц в каталоге, которые соответствуют шаблонам схемы и имен таблиц, а также заданным типам. **Схемой** называется группа связанных таблиц и полномочий доступа. **Каталогом** называется связанная группа схем. Параметры *catalog* и *schemaPattern* могут быть пустыми строками(" ") для извлечения таблиц без каталога и схемы либо иметь значение null, если нужно возвращать таблицы без учета каталога или схемы. Массив types содержит имена типов таблиц: TABLE, VIEW, ALIAS и т.д. Если для types задано значение null, то будут

возвращены таблицы всех типов. Результат выполнения запроса содержит 5 столбцов, которые имеют тип String и показаны в таблице 4.

Таблица 4

1	TABLE_CAT	Каталог таблиц (значение может быть равно null)
2	TABLE_SCHEM	Схема (значение может быть равно null)
3	TABLE_NAME	Имя таблицы
4	TABLE_TYPE	Тип таблицы
5	REMARKS	Комментарии для таблицы

Если нас интересует имя таблицы, т.е. третий столбец, то нужно вызвать метод *mrs.getString(3).*

Сведения о структуре набора результатов хранятся в классе *ResultSetMetaData.*

*ResultSet rs = statement.executeQuery("SELECT * FROM item");*

ResultSetMetaData rsmd= rs.getMetaData();

После получения результатов выполнения запроса можно определить количество столбцов методом *getColumnCount()*, имена столбцов *getColumnLabel(int column)* и т.д.